CSC1029 OO Programming

Lecture 25 – Fundamental Data Structures

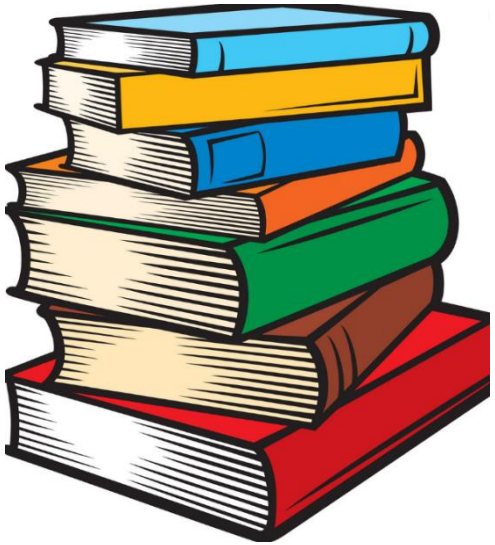
QUEEN'S UNIVERSITY BELFAST

What's to come in todays lecture?

- Introduction to the basic data structures:
  - Linked Lists
  - Stacks
  - Queues
- Implementation of Linked Lists

Basic Data Structures

Lists



Stacks

Queues

Singly Linked Lists

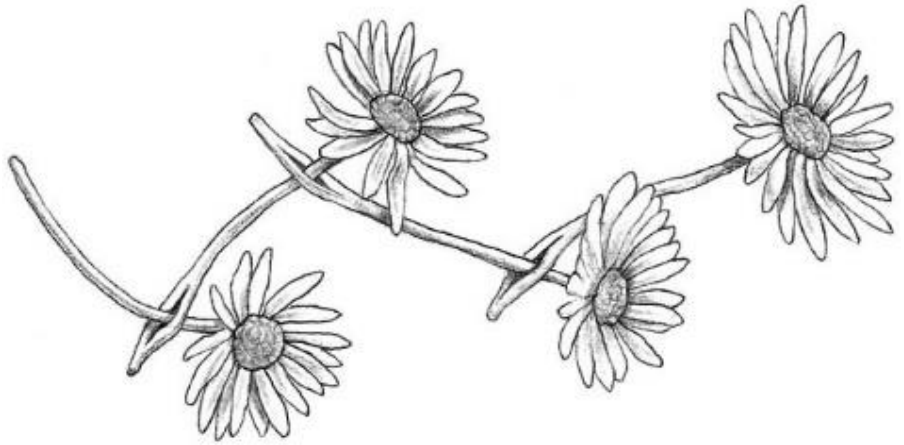
A ***linked list*** is an example of a ***dynamic data structure,*** used for storing and managing data of a given type.

Such structures typically grow in size (in terms of resource requirements) as data is added to the structure and will correspondingly reduce in size as data is removed.

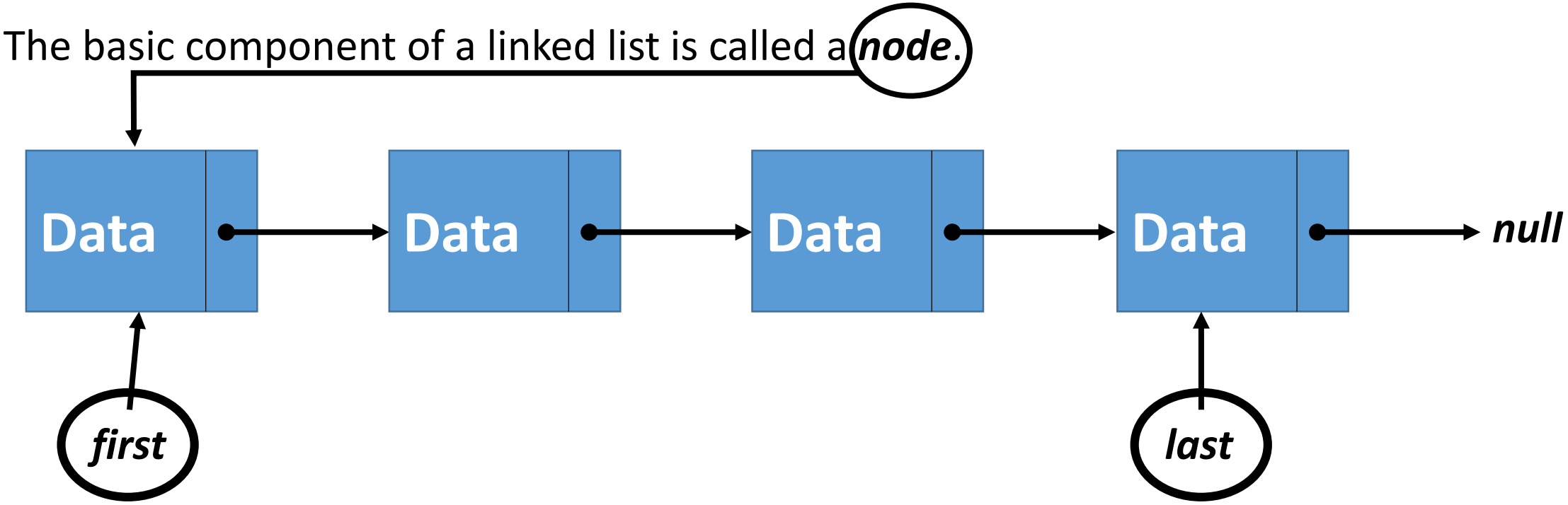
Unlike array structures (which are fixed in size), dynamic data structures can continue to grow in size, providing the resources are available.

Array Lists (internally) make use of such dynamic data structures of part of their implementation.

Singly Linked Lists



Singly Linked Lists



Each ***node*** in a ***singly-linked list*** has 2 components: a ***data*** object reference and a link to the ***'next' node*** in the list. A typical list can have 0, 1 or more nodes in sequence.

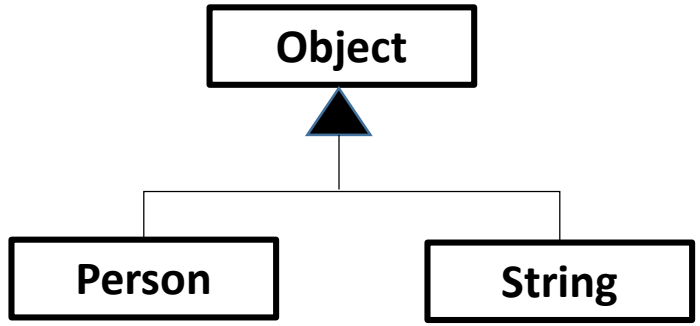
An entire list can be maintained with a reference to the top/head/first of the list.

It's common to have more than one reference to manage a list.

Singly Linked Lists

LinkedList
-first:Node -last:Node -numElements:int
+LinkedList() +add(Object) +add(Object,int):boolean +remove():Object +remove(int):Object +toString():String +size():int

```
private class Node {
    public Object data;
    public Node next;
}
```



Singly Linked Lists

LinkedList
-first:Node -last:Node -numElements:int
+LinkedList() +add(Object) +add(Object,int):boolean +remove():Object +remove(int):Object +toString():String +size():int

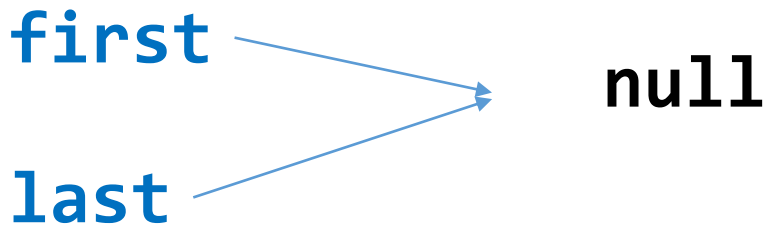
```
LinkedList myList =
new LinkedList();
```

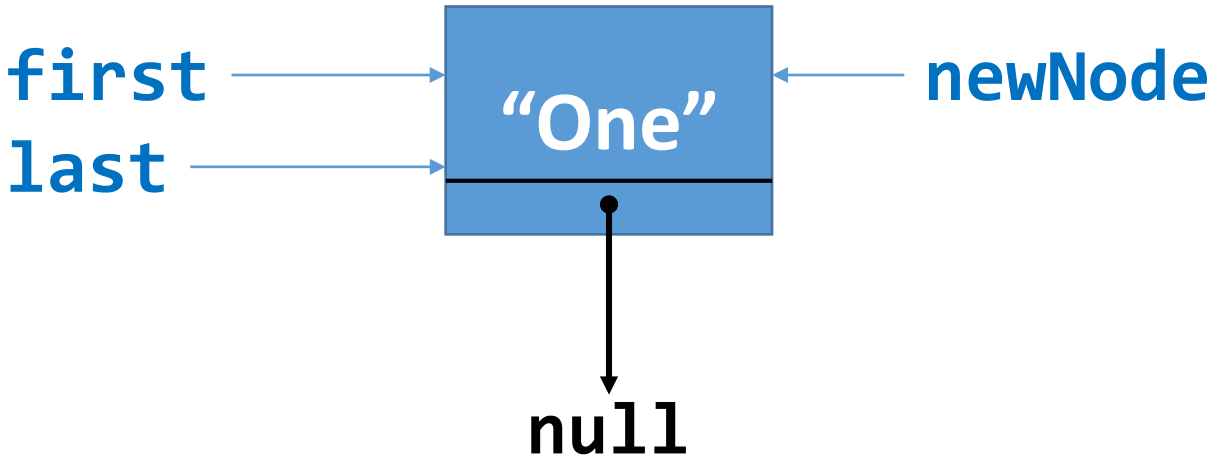
Singly Linked Lists (add to end of list)

myList.add("One");

```
public void add(Object element) {
    if (element == null)
        return;
    Node newNode = new Node();
    newNode.data = element;
    if (this.first == null) {
        this.first = newNode;
    } else {
        this.last.next = newNode;
    }
    this.last = newNode;
    this.numElements++;
}
```

Before





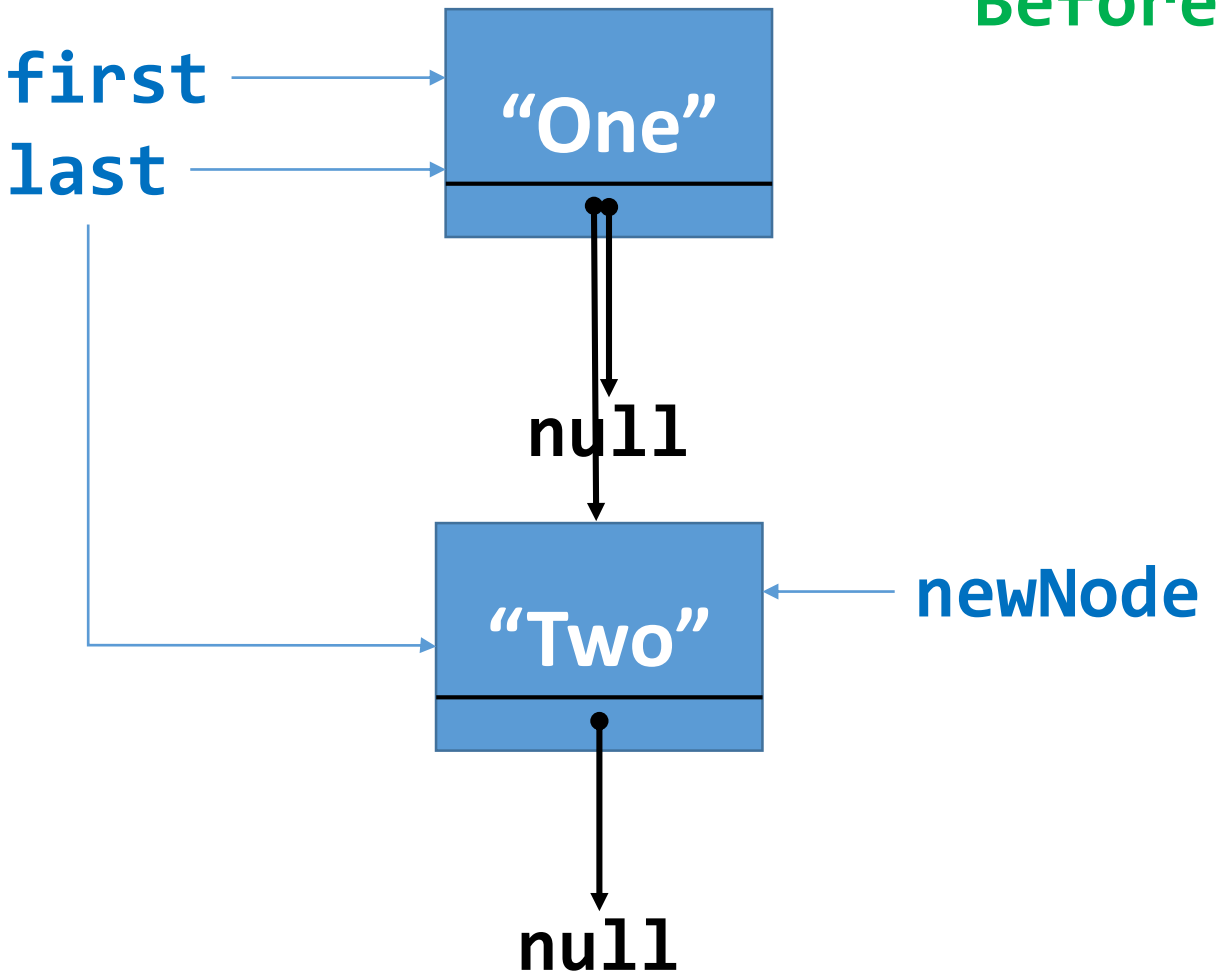
After

Singly Linked Lists (add to end of list)

myList.add("Two");

```
public void add(Object element) {
    if (element == null)
        return;
    Node newNode = new Node();
    newNode.data = element;
    if (this.first == null) {
        this.first = newNode;
    } else {
        this.last.next = newNode;
    }
    this.last = newNode;
    this.numElements++;
}
```

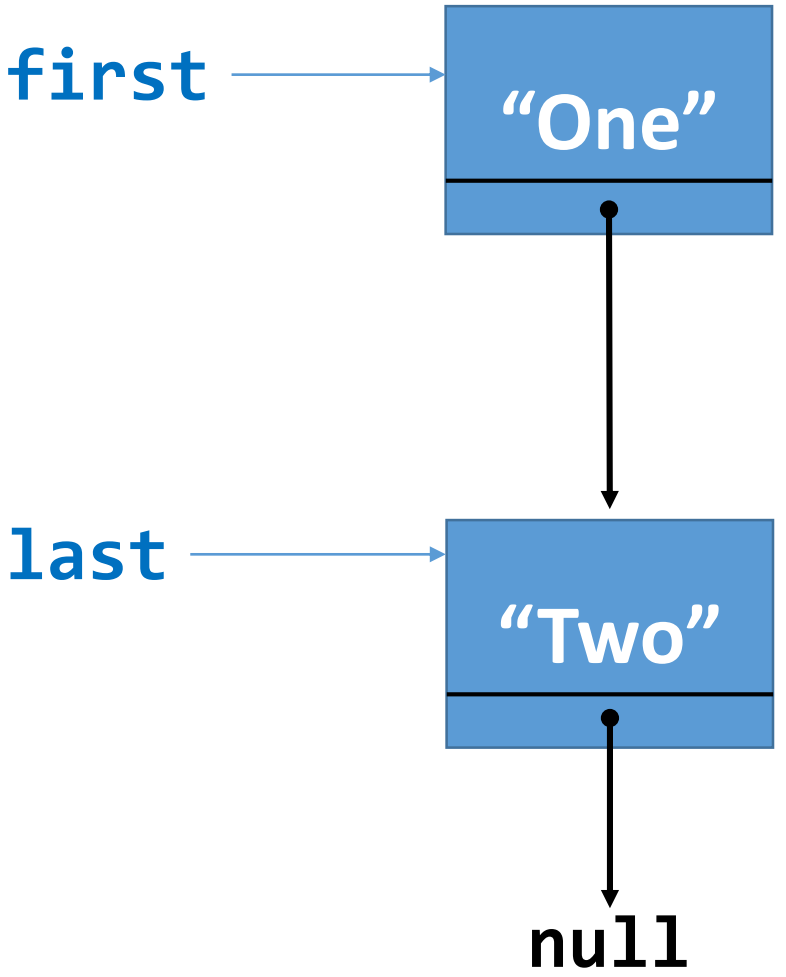
Before



Singly Linked Lists (add to end of list)

myList.add("Two");

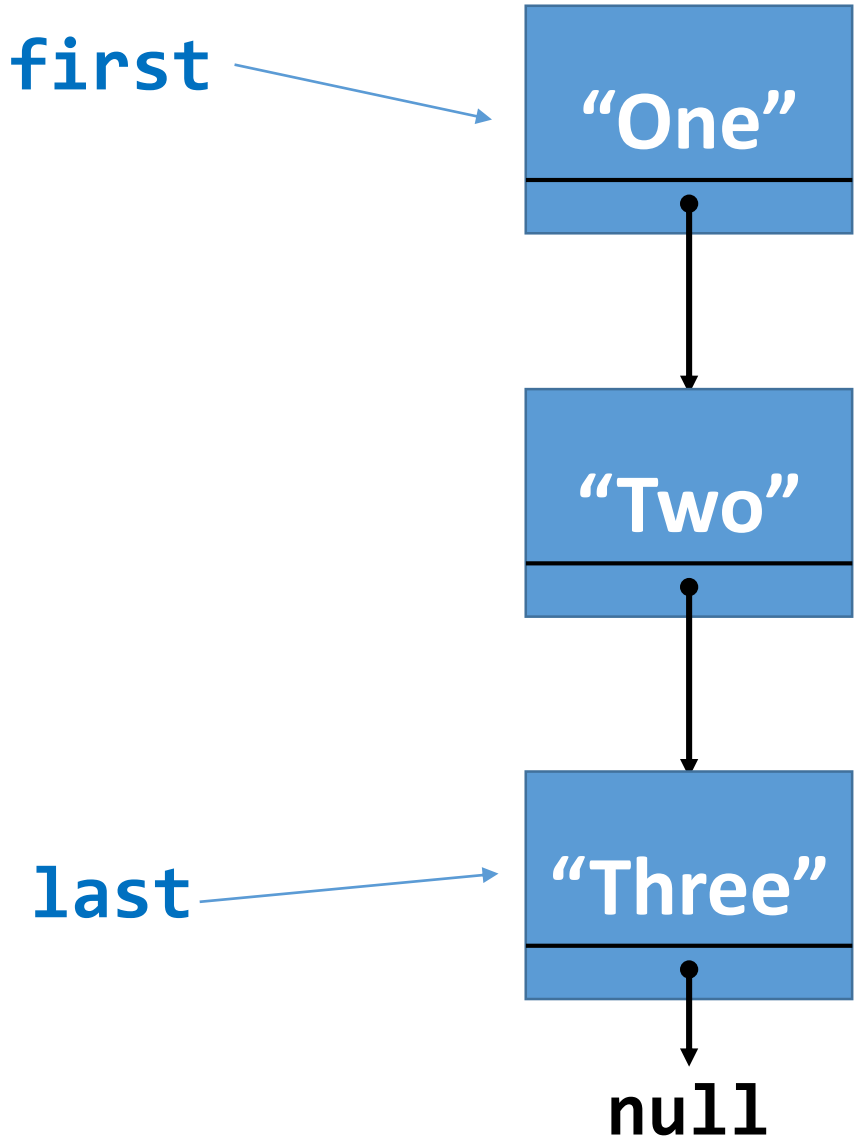
```
public void add(Object element) {
    if (element == null)
        return;
    Node newNode = new Node();
    newNode.data = element;
    if (this.first == null) {
        this.first = newNode;
    } else {
        this.last.next = newNode;
    }
    this.last = newNode;
    this.numElements++;
}
```



After

Singly Linked Lists (add to end of list)

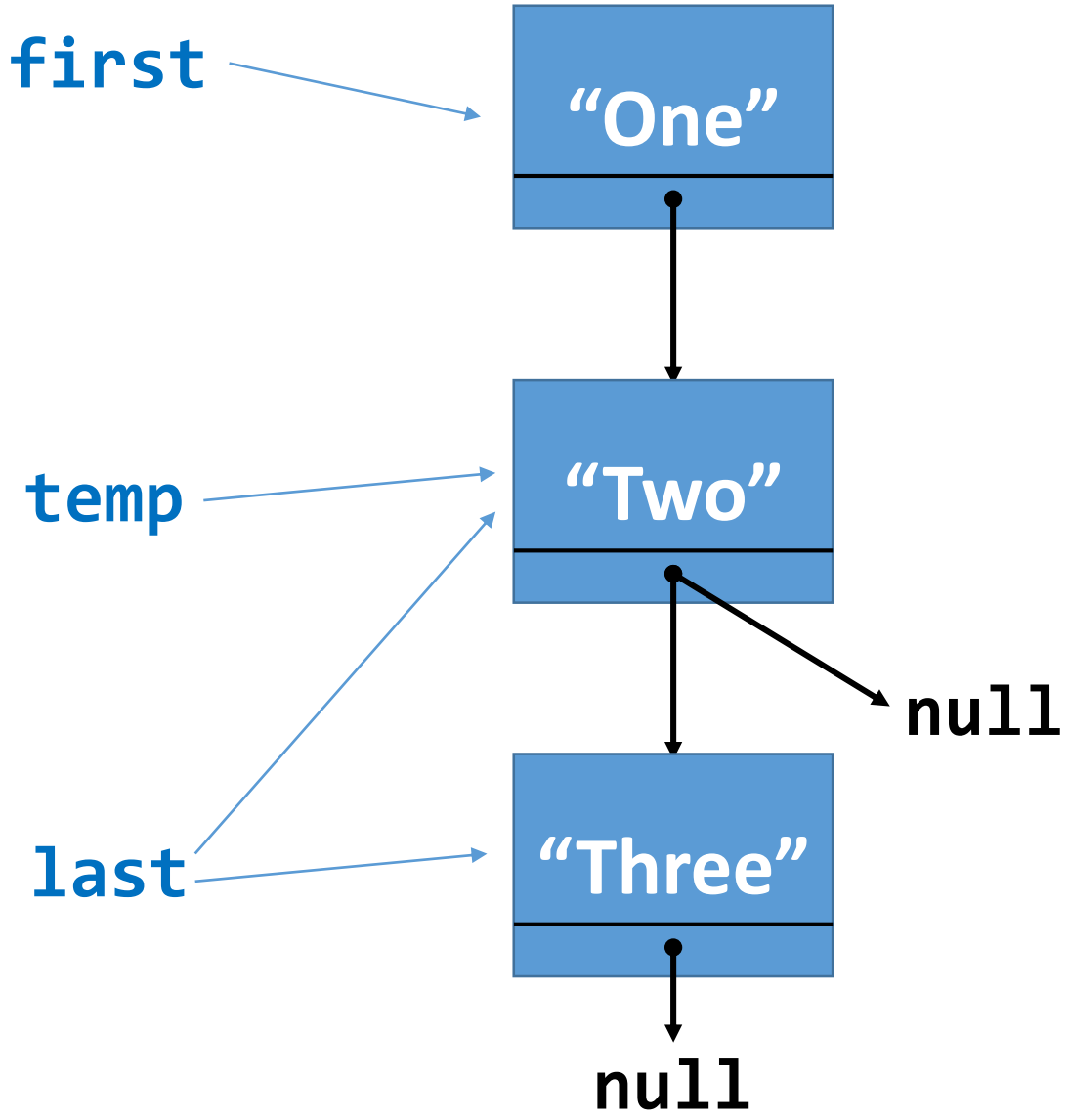
```
LinkedList myList =
new LinkedList();

myList.add("One");

myList.add("Two");

myList.add("Three");
```



Singly Linked Lists (remove from end)

myList.remove();

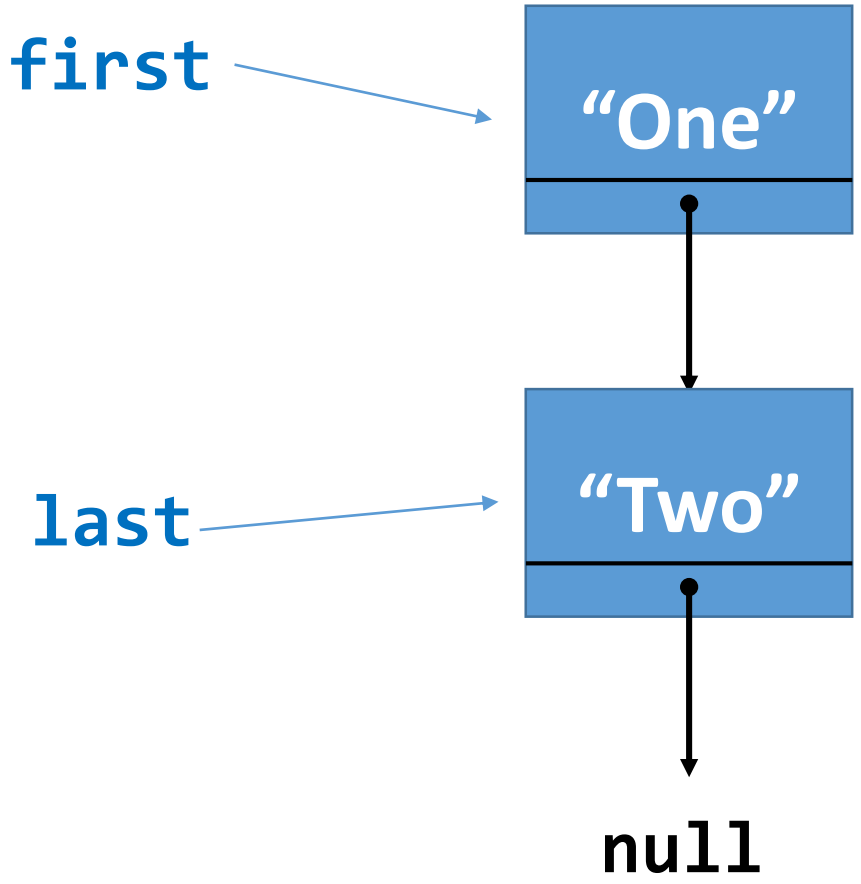
```
public Object remove() {
    if (numElements == 0) {
        return null;
    }
    Object res = this.last.data;
    if (numElements == 1) {
        this.first = null;
        this.last = null;
    } else {
        Node temp = select(numElements-2);
        this.last = temp;
        this.last.next = null;
    }
    this.numElements--;
    return res;
}
```



Singly Linked Lists (remove from end)

myList.remove();

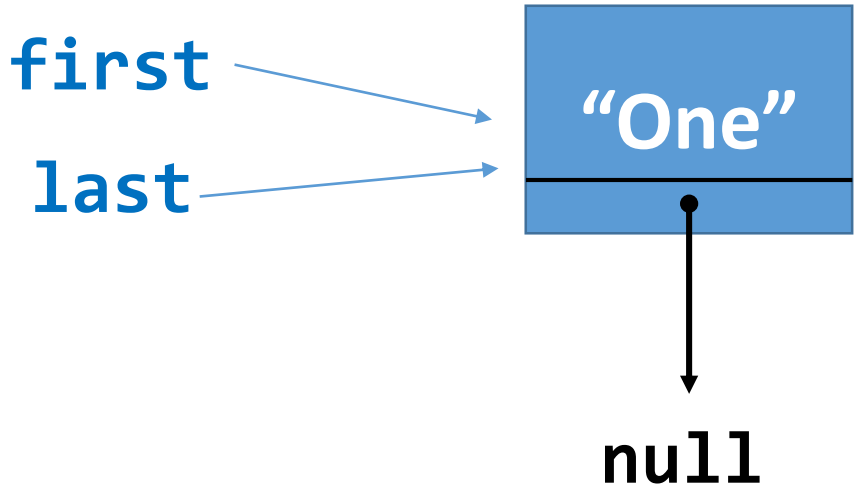
```
public Object remove() {
   if (numElements == 0) {
      return null;
   }
   Object res = this.last.data;
   if (numElements == 1) {
      this.first = null;
      this.last = null;
   } else {
      Node temp = select(numElements-2);
      this.last = temp;
      this.last.next = null;
   }
   this.numElements--;
   return res;
}
```



Singly Linked Lists (remove from end)

myList.remove();

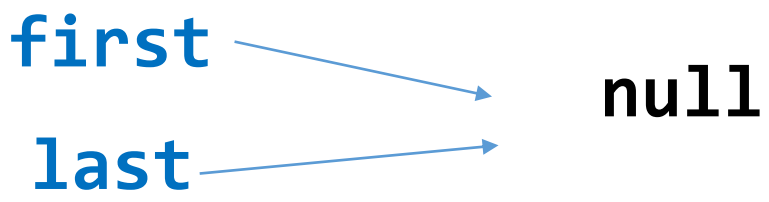
```
public Object remove() {
   if (numElements == 0) {
      return null;
   }
   Object res = this.last.data;
   if (numElements == 1) {
      this.first = null;
      this.last = null;
   } else {
      Node temp = select(numElements-2);
      this.last = temp;
      this.last.next = null;
   }
   this.numElements--;
   return res;
}
```



Singly Linked Lists (remove from end)

myList.remove();

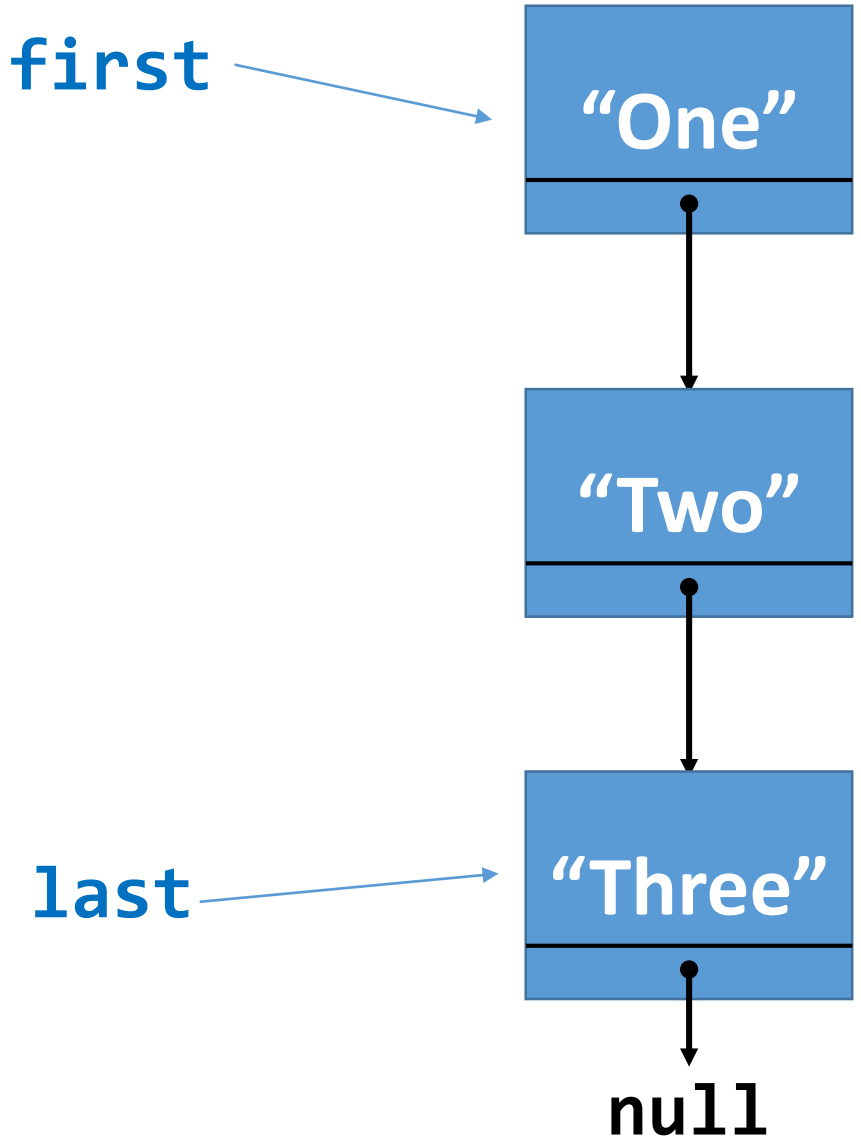
```
public Object remove() {
   if (numElements == 0) {
      return null;
   }
   Object res = this.last.data;
   if (numElements == 1) {
      this.first = null;
      this.last = null;
   } else {
      Node temp = select(numElements-2);
      this.last = temp;
      this.last.next = null;
   }
   this.numElements--;
   return res;
}
```



Singly Linked Lists (remove from index)

myList.remove(0);

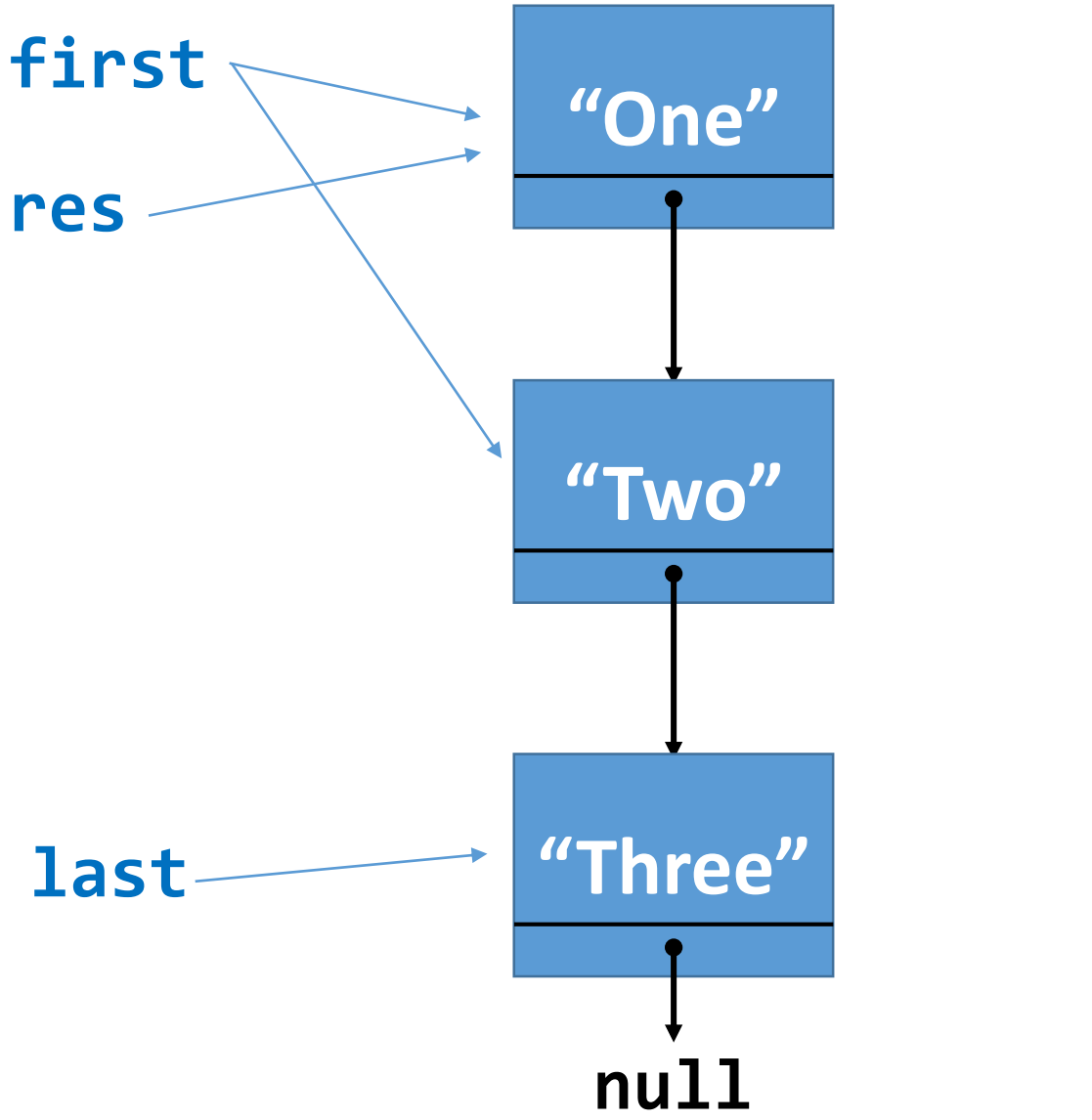
```
public Object remove(int position) {
    if (numElements == 0  || position<0 ||
          position>=numElements) {
       return null;
    }
    Object res;
    if (position == 0) {
       // remove from start
    }
    else if (position == numElements-1) {
       // remove from end
    }
    else {  // remove from the middle
    }
    this.numElements--;
    return res;
}
```



Singly Linked Lists (remove from index)

myList.remove(0);

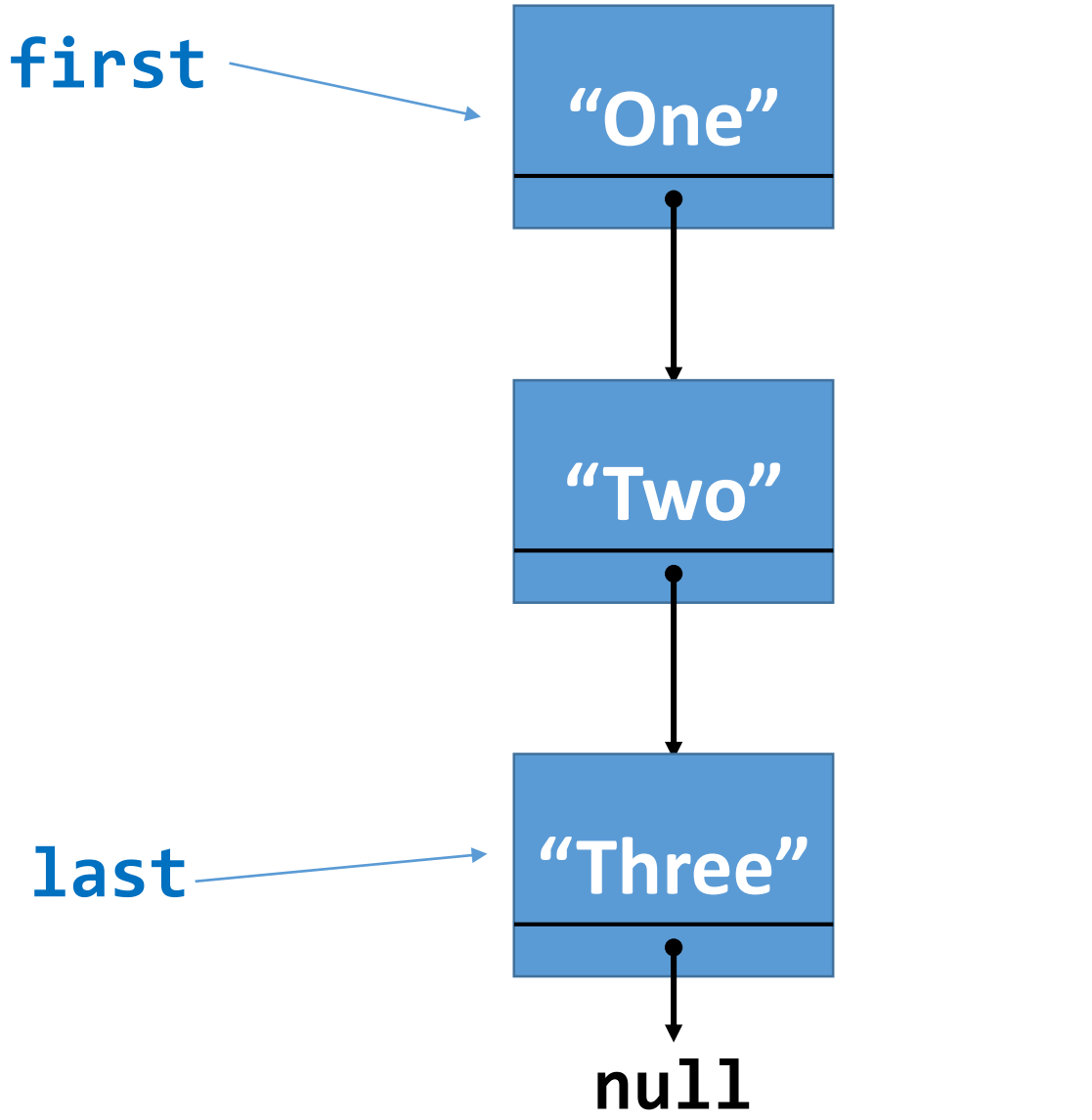
```
Object res;
...

// remove from start
res = this.first.data;
this.first = this.first.next;
if (this.first == null)
   this.last = null;
```



Singly Linked Lists (remove from index)

```
myList.remove(2);
```

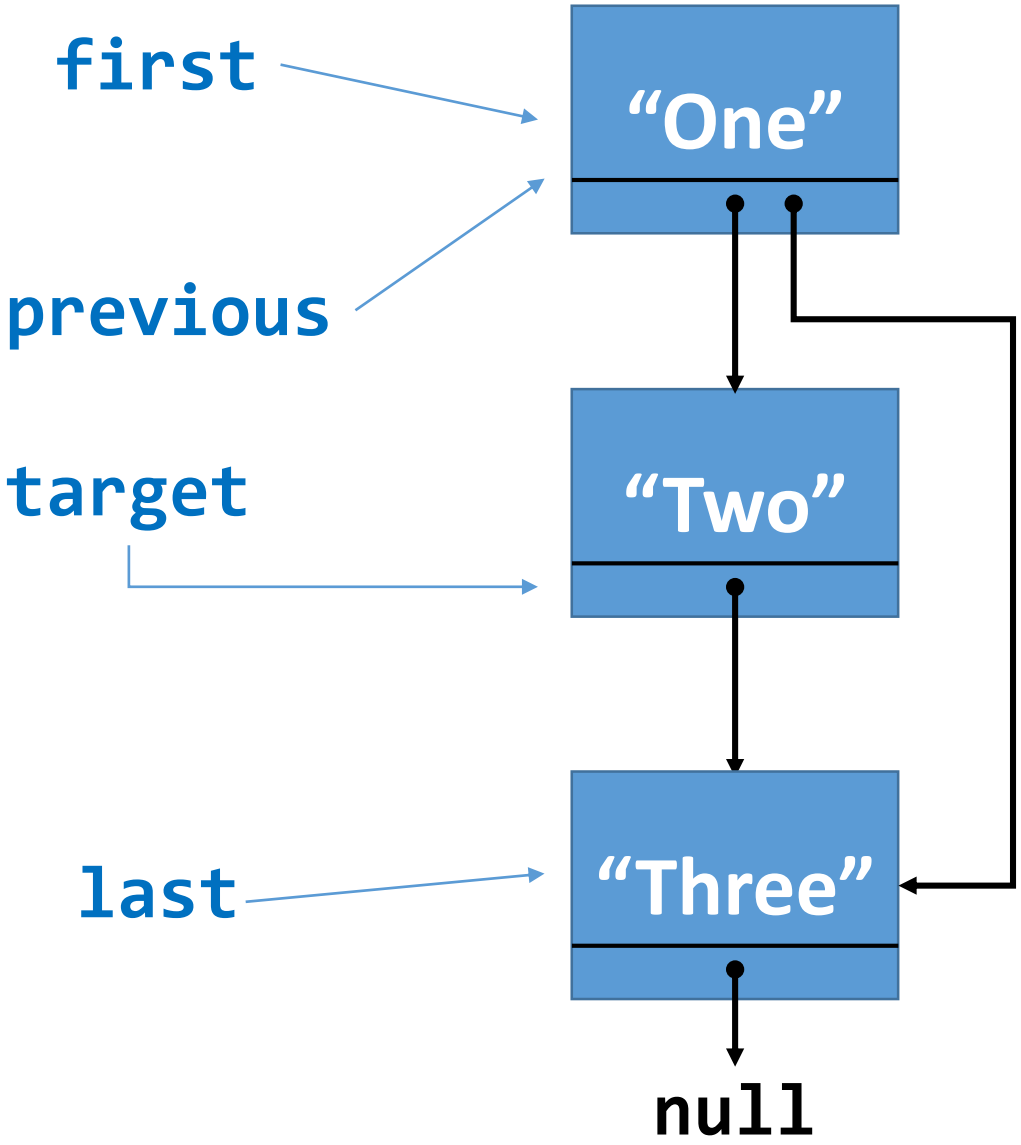
```
Object res;
...

// remove from end
res = remove();
```



Singly Linked Lists (remove from index)

```
myList.remove(1);
```

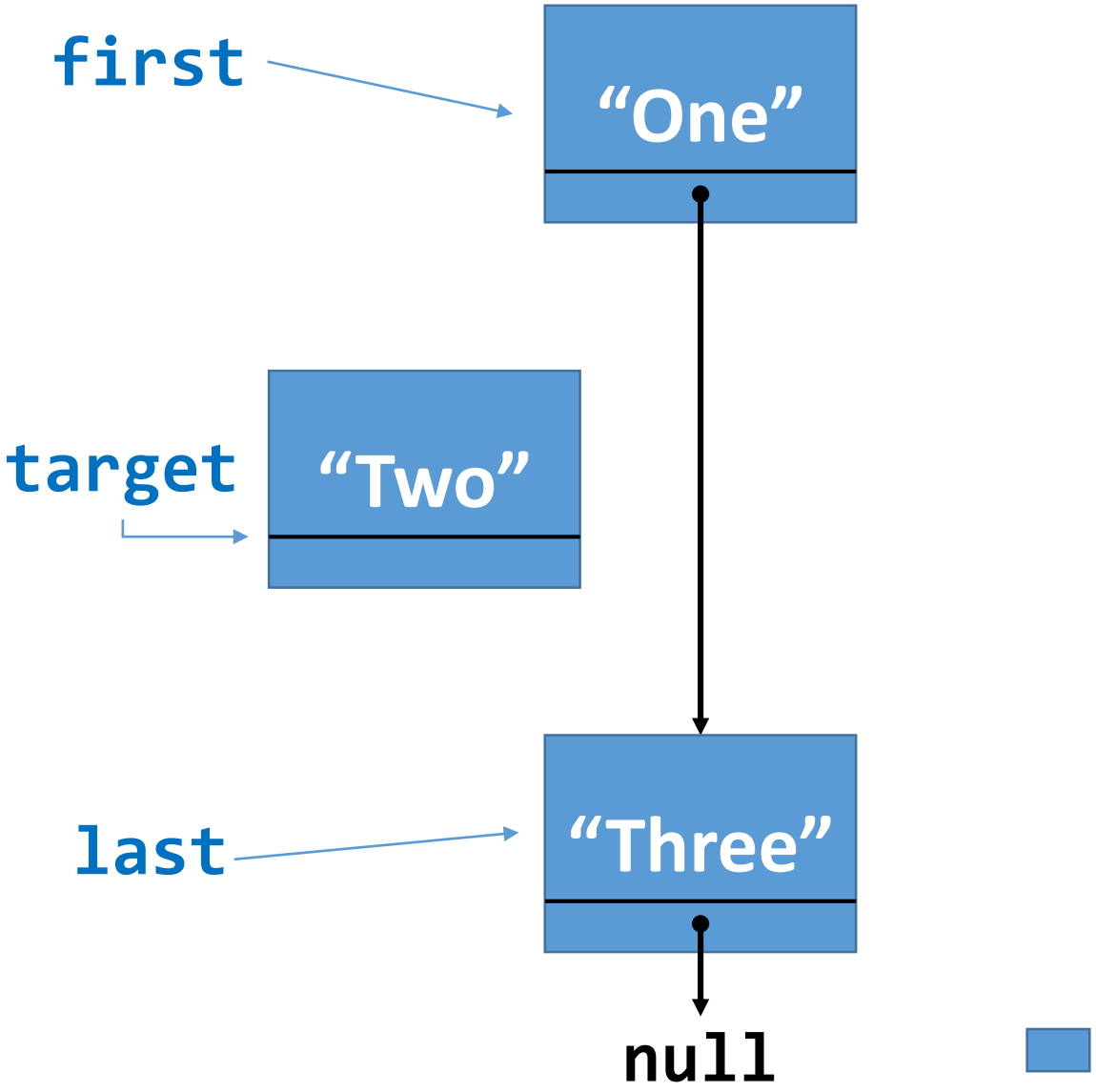
```
Object res;
...

// remove from the middle
Node previous = select(position - 1);
Node target = previous.next;
previous.next = target.next;
res = target.data;
```



Singly Linked Lists (remove from index)

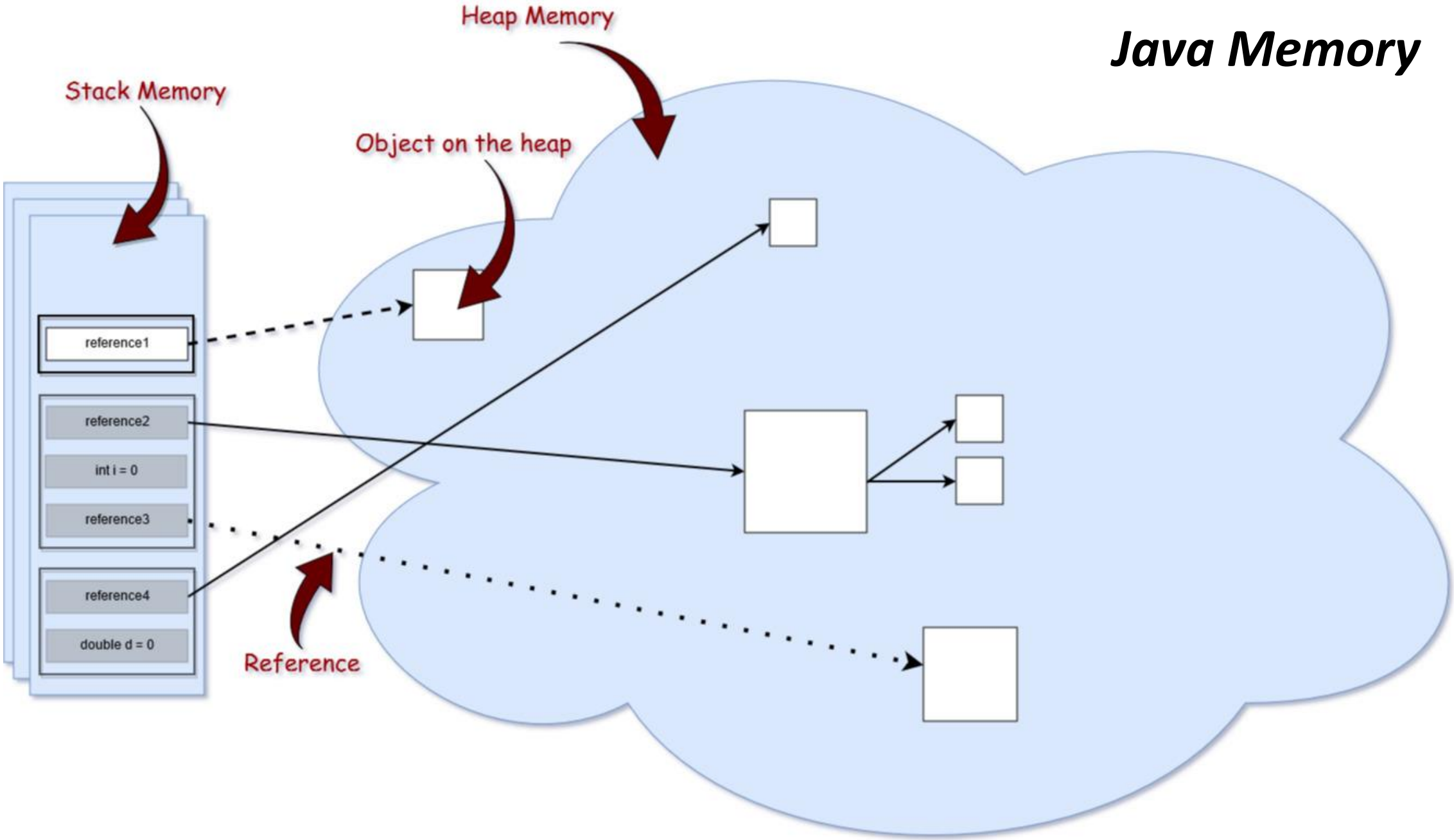
```
myList.remove(1);
```

```
Object res;
...

// remove from the middle
Node previous = select(position - 1);
Node target = previous.next;
previous.next = target.next;
res = target.data;
```



Memory Organisation

Java Memory

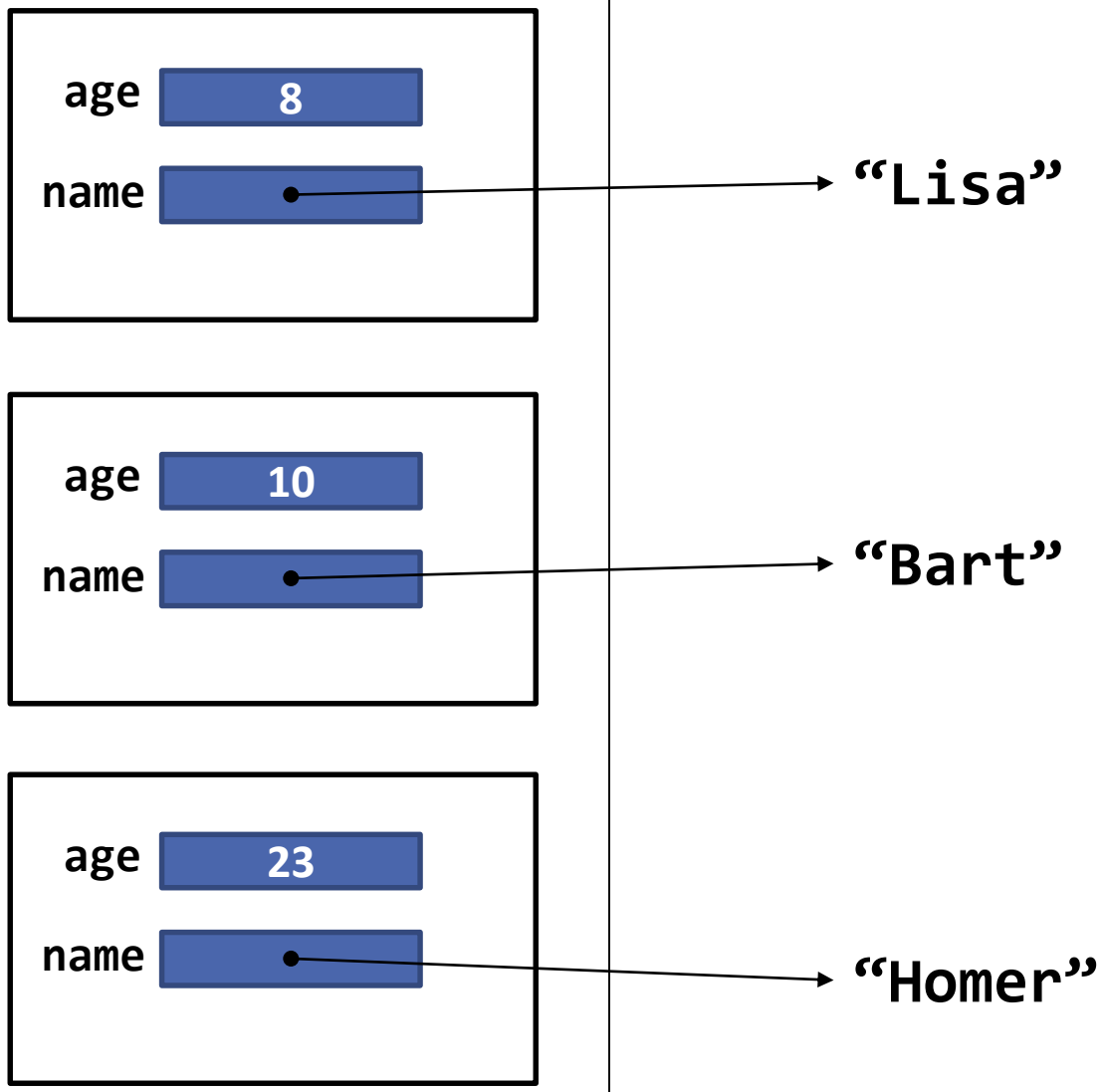

Heap Memory
Stack Memory
Object on the heap
reference1
reference2
int i = 0
reference3
reference4
double d = 0
Reference

Code

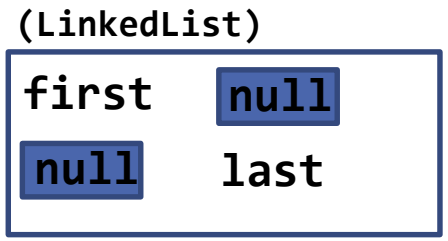
```
public static void method2() {
    int age = 8;
    String name = “Lisa”;
}

public static void method1() {
    int age = 10;
    String name = “Bart”;
    method2();
}

public static void main(String args[]) {
    int age = 23;
    String name = “Homer”;
    method1();
}
```

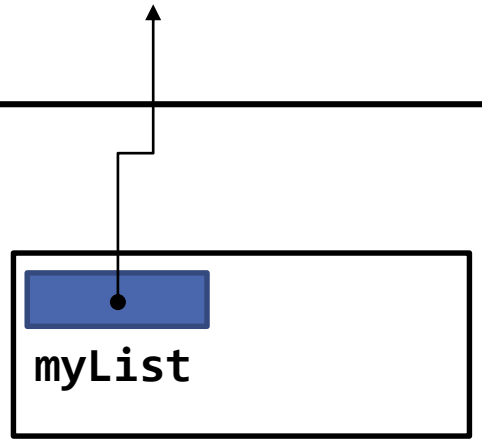
Stack Memory

Heap Memory



Heap Memory

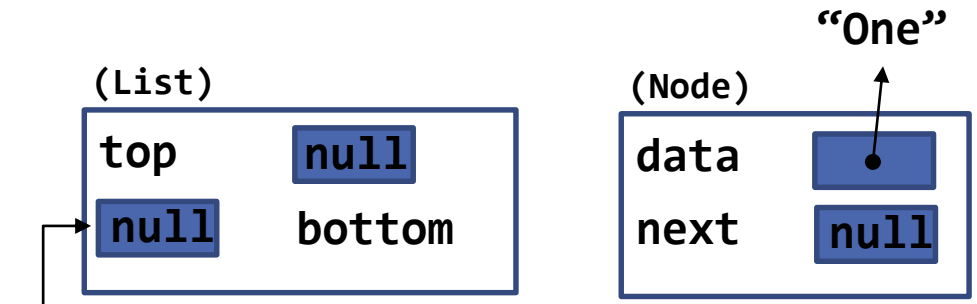





```
public static void main(String args[]) {
    LinkedList myList = new LinkedList();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

Stack Memory

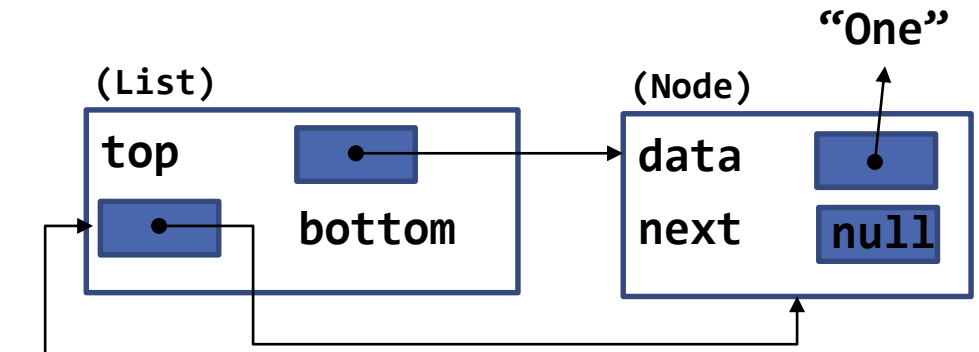
Heap Memory




```
public static void main(String args[]) {
    List myList = new List();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

Stack Memory

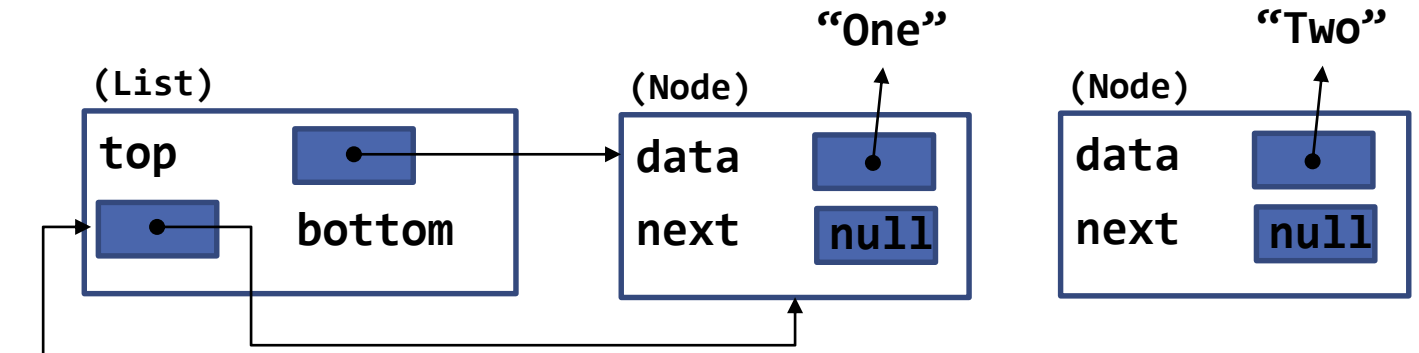
Heap Memory

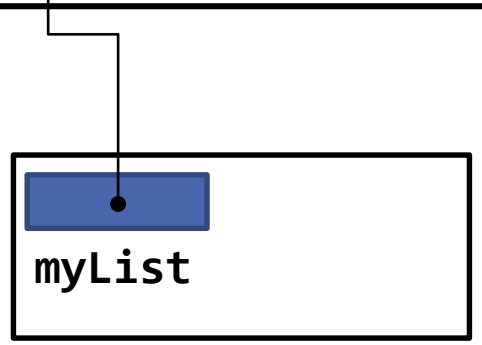



```
public static void main(String args[]) {
    List myList = new List();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

Stack Memory

Heap Memory

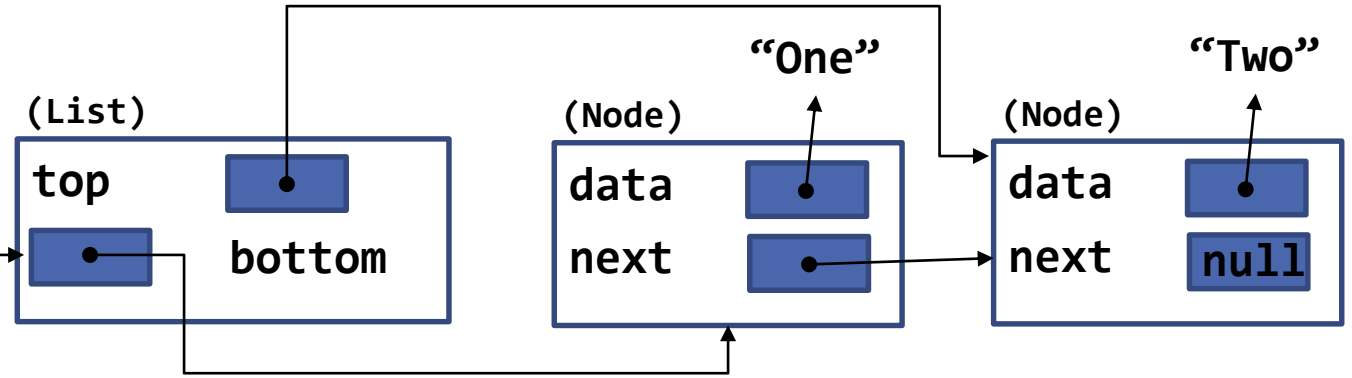


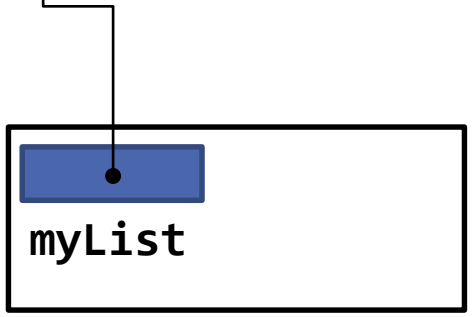



```
public static void main(String args[]) {
    List myList = new List();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

Stack Memory

Heap Memory

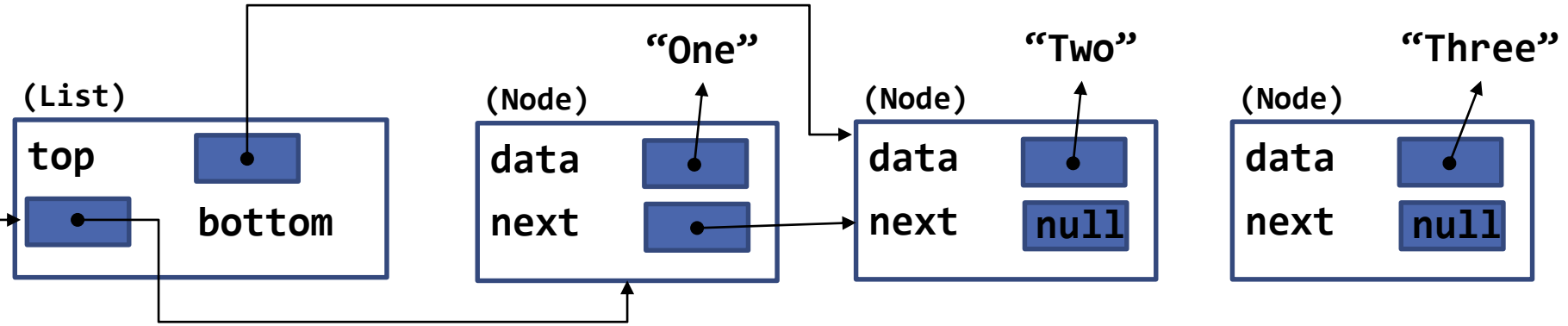





```
public static void main(String args[]) {
    List myList = new List();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

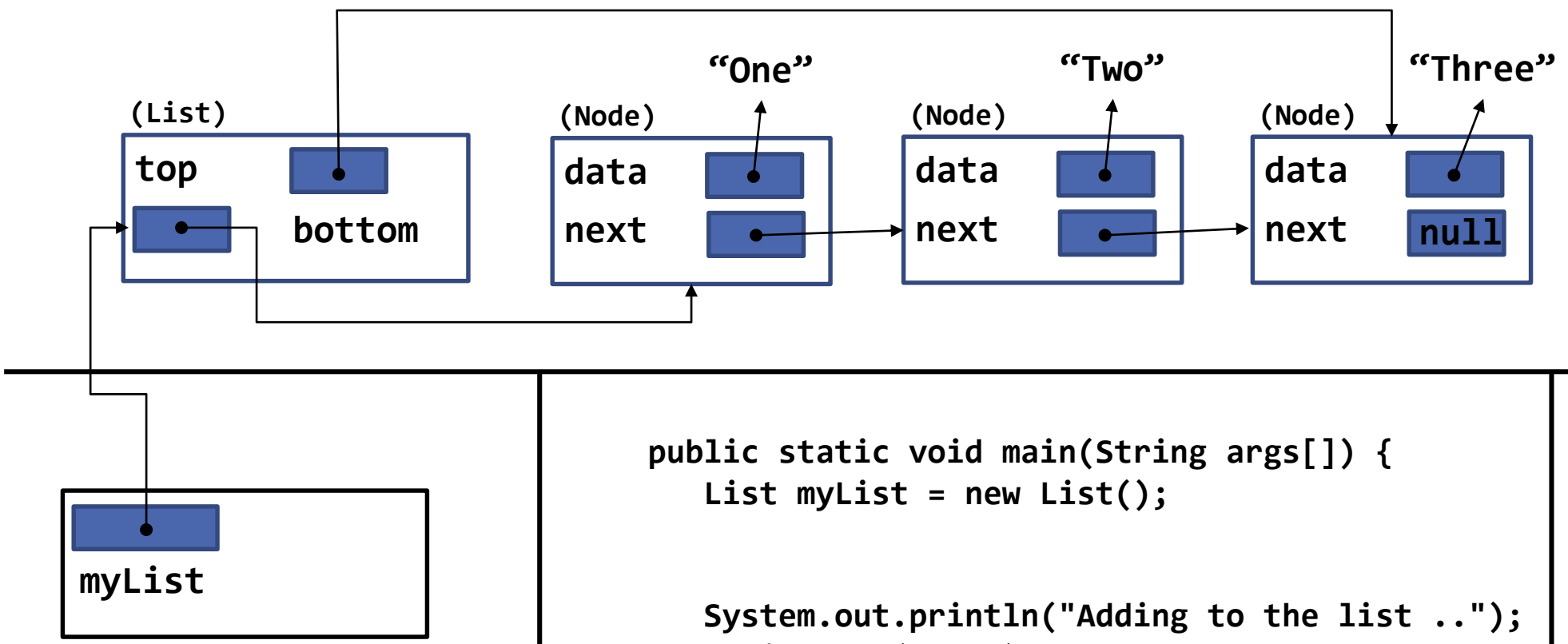
Stack Memory

Heap Memory

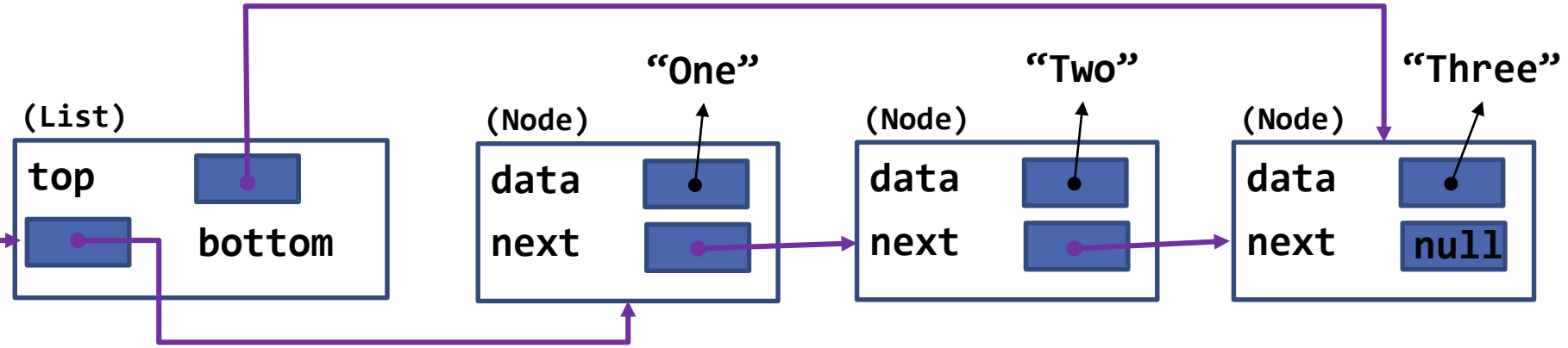



```
public static void main(String args[]) {
    List myList = new List();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

Stack Memory


Heap Memory
(List)
top
bottom
(Node)
data
next
"One"
(Node)
data
next
"Two"
(Node)
data
next
null
"Three"
myList
public static void main(String args[]) {
List myList = new List();
System.out.println("Adding to the list ..");
myList.add("One");
myList.add("Two");
myList.add("Three");
System.out.println(myList);
}
Stack Memory

Heap Memory




```
public static void main(String args[]) {
    List myList = new List();


    System.out.println("Adding to the list ..");
    myList.add("One");

    myList.add("Two");

    myList.add("Three");

    System.out.println(myList);
}
```

Output

```
One
Two
Three
```

Stack Memory

What's Next?

Fundamental Data Structures